ELO320 - Estructuras de Datos y Algoritmos
HeapSort
Colas de Prioridad

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

HeapSort

Es un algoritmo de ordenamiento de complejidad $\mathcal{O}(n \log n)$, que combina la complejidad de MergeSort (se basa en un árbol binario) y la inserción *in-place* de InsertSort.

HeapSort se basa en una estructura de datos llamada **Heap** (que no es la memoria):

- Árbol binario.
- Árbol completo $\rightarrow$ $k-1$ niveles llenos, nivel k parcial (izq. a der.).
- Padres $>$ Hijos $\rightarrow$ MAX-HEAP.
- Padres $<$ Hijos $\rightarrow$ MIN-HEAP.

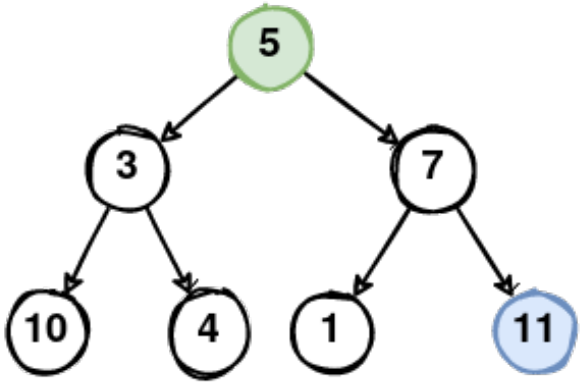
Nosotros **visualizaremos** un arreglo como un Heap, vale decir, **no crearemos un árbol**. Trabajaremos en el arreglo como si lo fuera.

HEAP

COMO SON LOS DATOS

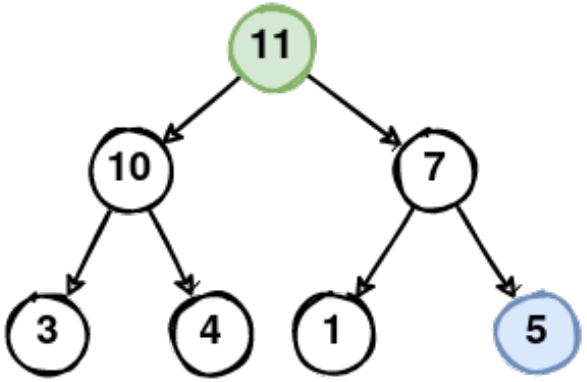
5	3	7	10	4	1	11

COMO SE VEN



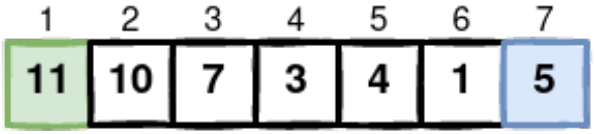
COMO DEBEN QUEDAR

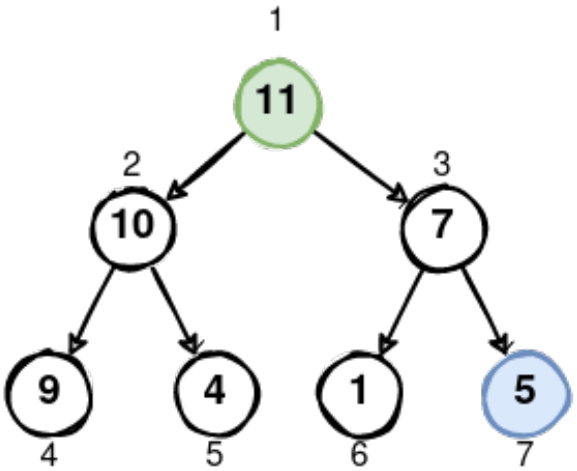
11	10	7	3	4	1	5



HeapSort: Heap

La estructura del árbol depende de los índices del arreglo. Así, la **visualización** es fácil de establecer.





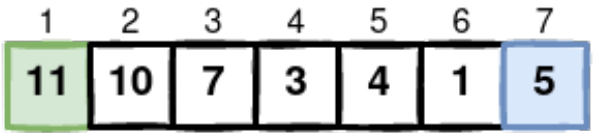
Noten, que estamos trabajando con índices a partir del **número 1**, para simplificar la implementación.

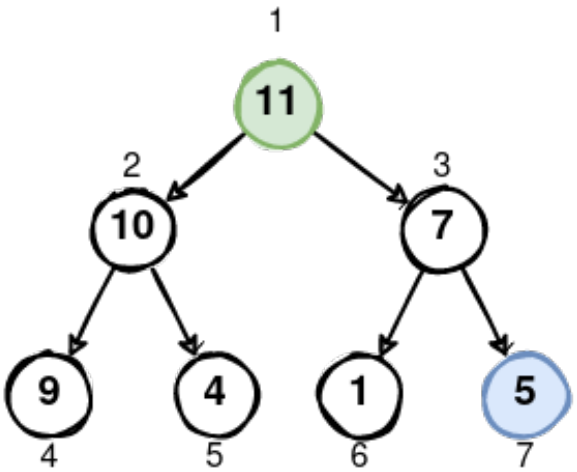
HeapSort: Heap

Funciones útiles

```
int padre(int i){ //padre del nodo con índice i
        return i/2; //div. entera (o usar floor de math.h)
}

int hijo_izq(int i){ //hijo izq del nodo con índice i
        return 2*i;
}

int hijo_der(int i){ //hijo der del nodo con índice i
        return (2*i)+1
}
```

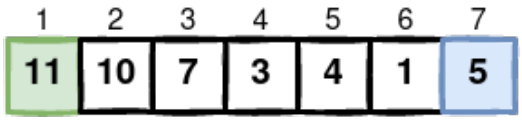
HeapSort: Heap

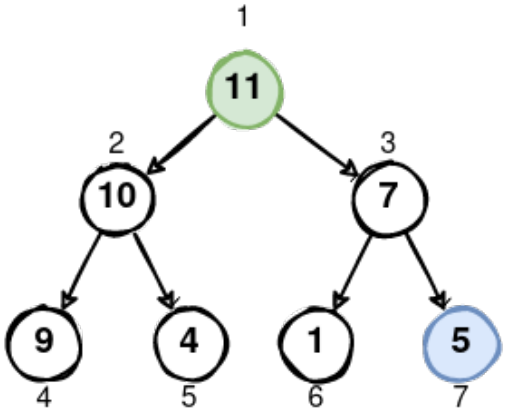


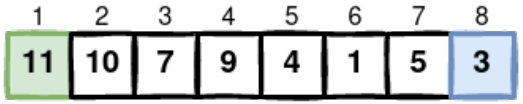


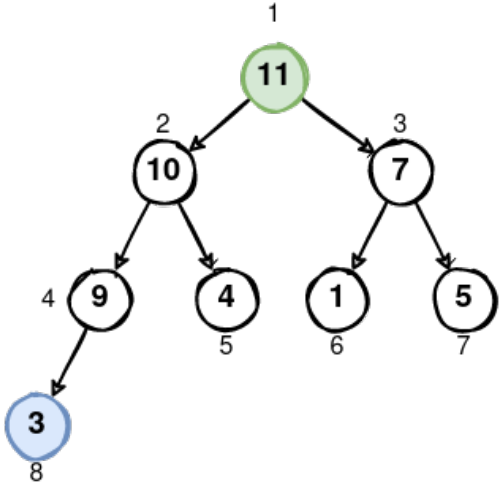
HeapSort: Heap

En esta clase nos centraremos en MAX-HEAP.





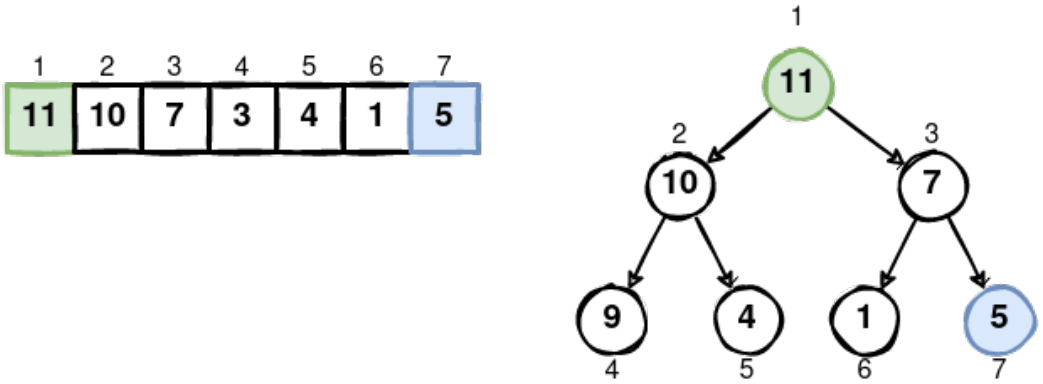


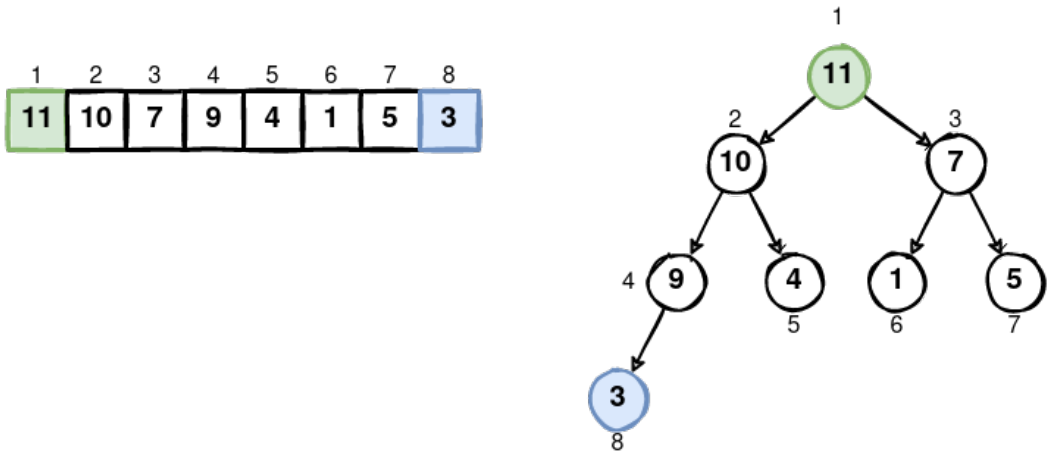


HeapSort: Heap

Noten que:

- El inicio del arreglo (raíz del árbol), será el mayor valor.
- El final del arreglo, será el nodo más a la derecha del último nivel (incompleto).



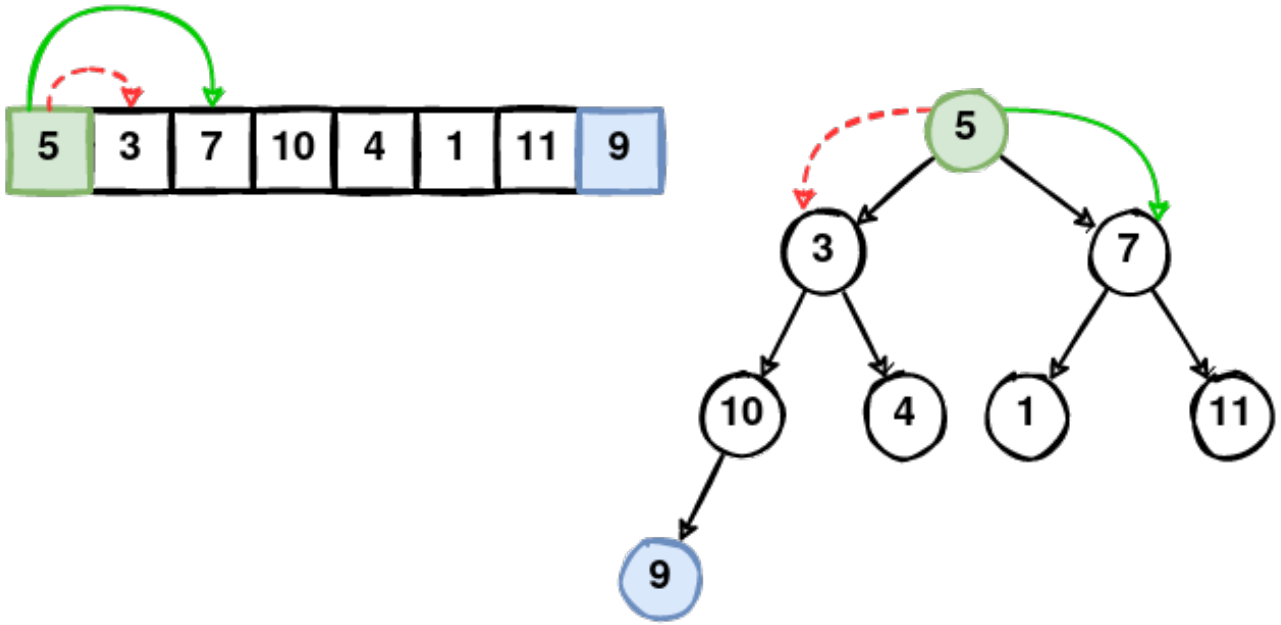


HeapSort: Componentes

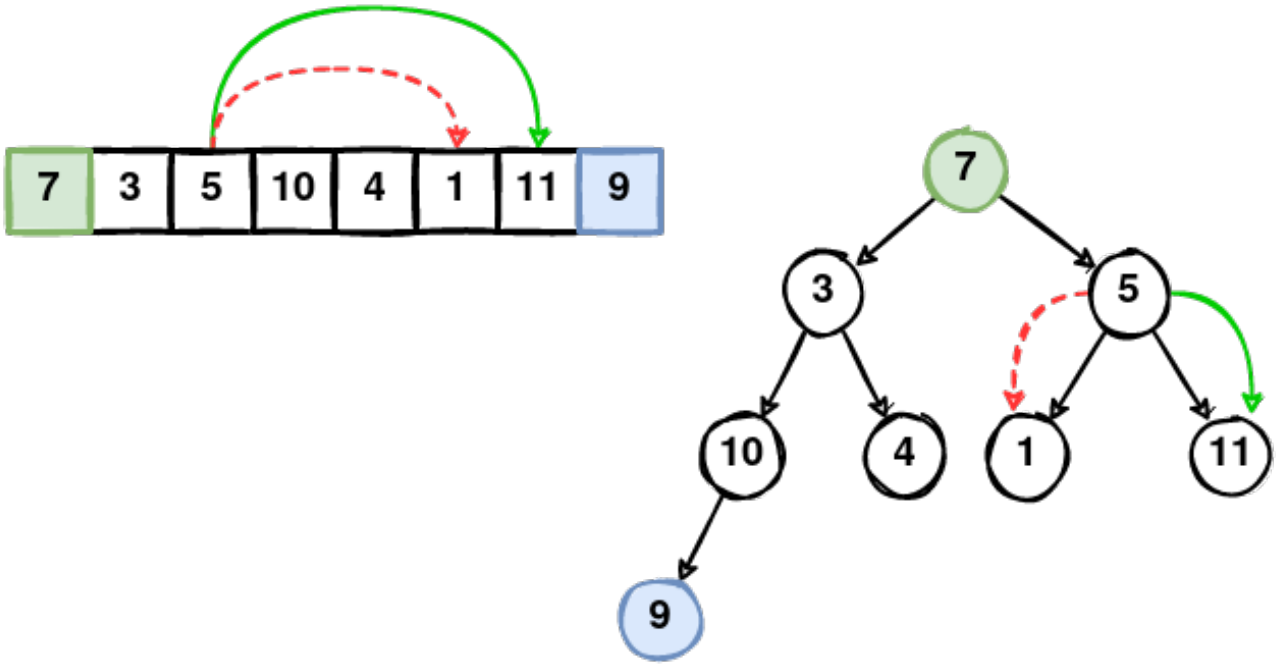
HeapSort se puede dividir en tres procedimientos:

- `heapify`: Función recursiva que permite mantener las reglas del heap. Es de complejidad $\mathcal{O}(\log n)$.
- `build_heap`: Función que permite dejar el arreglo completo en formato MAX-HEAP o MIN-HEAP. Es lineal ($\mathcal{O}(n)$).
- `HeapSort`: Procedimiento que ordena los datos utilizando las características del heap establecido. Es $\mathcal{O}(n \log n)$.

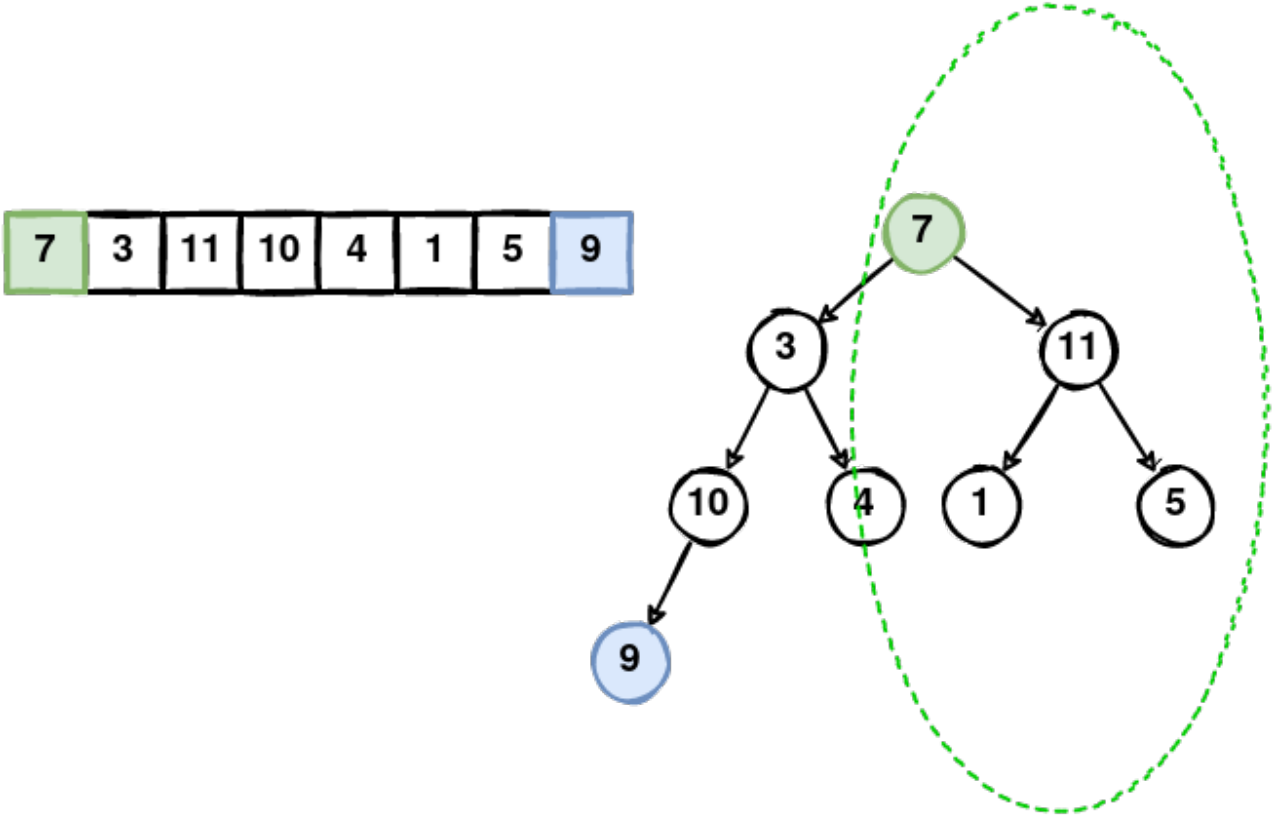
HeapSort: Heapify



HeapSort: Heapify



HeapSort: Heapify

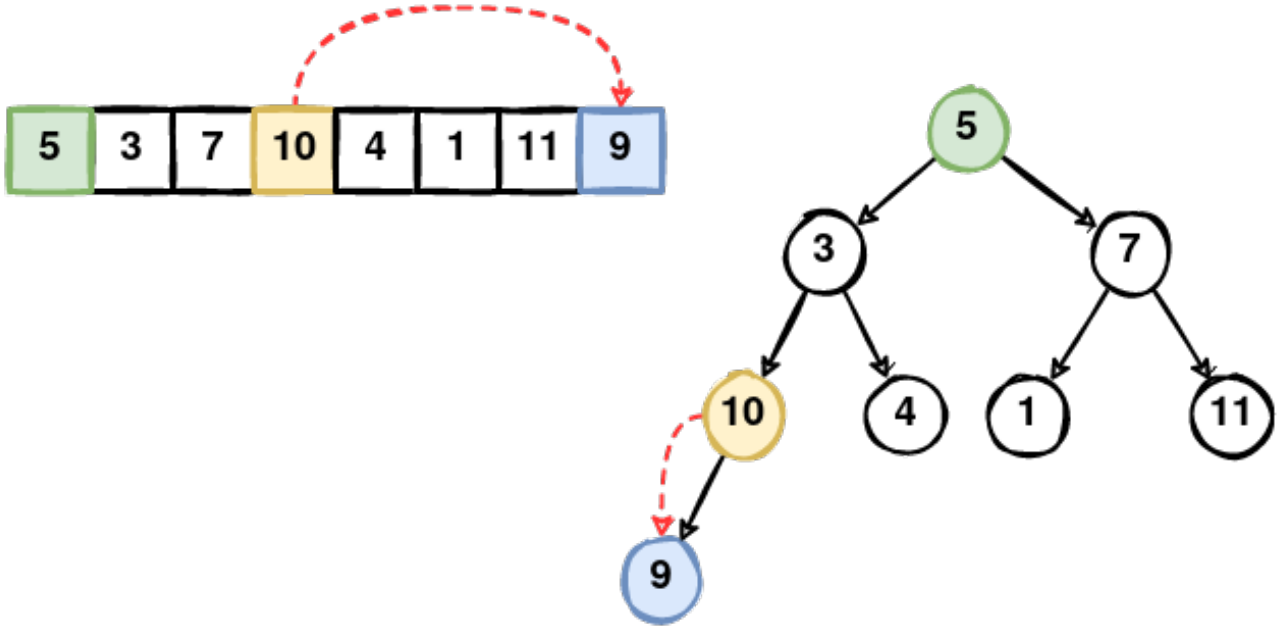


HeapSort: Heapify

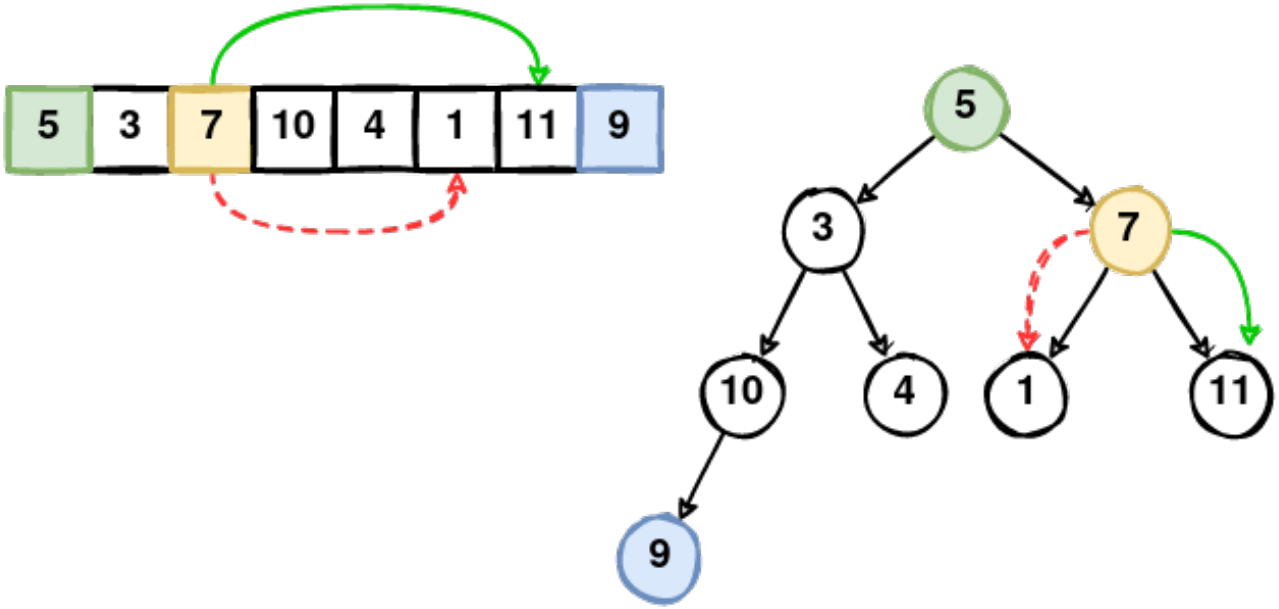
Heapify

```
void heapify(int arr[], int i, int tam){ //arreglo, índice, tamaño
    int mayor = i; //al inicio el nodo es el mayor
    int l = hijo_izq(i);
    int r = hijo_der(i);

    if (l < tam && arr[l] > arr[mayor]) //si no nos pasamos del largo
        mayor = l;                      //y el padre es menor que el hijo izq

    if (r < tam && arr[r] > arr[mayor]) //luego la comparación con el hijo der
        mayor = r;

    if (mayor != i){ //intercambio
        int temp = arr[i];
        arr[i] = arr[mayor];
        arr[mayor] = temp;

        heapify(arr, mayor, tam); //recursividad sub-árbol
    }
}
```

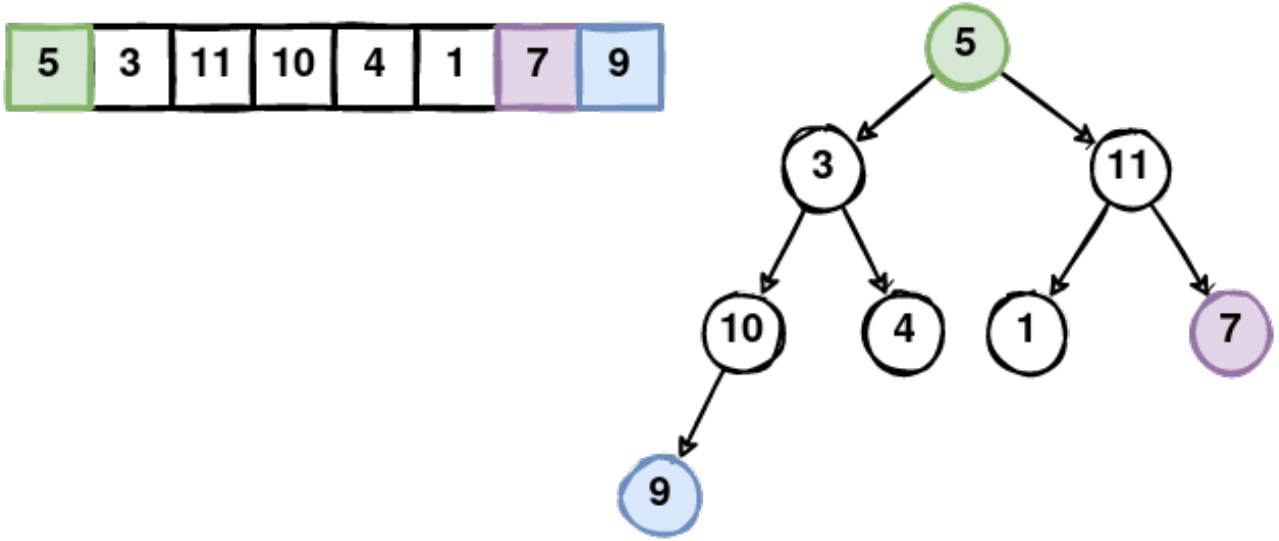
HeapSort: Build Heap



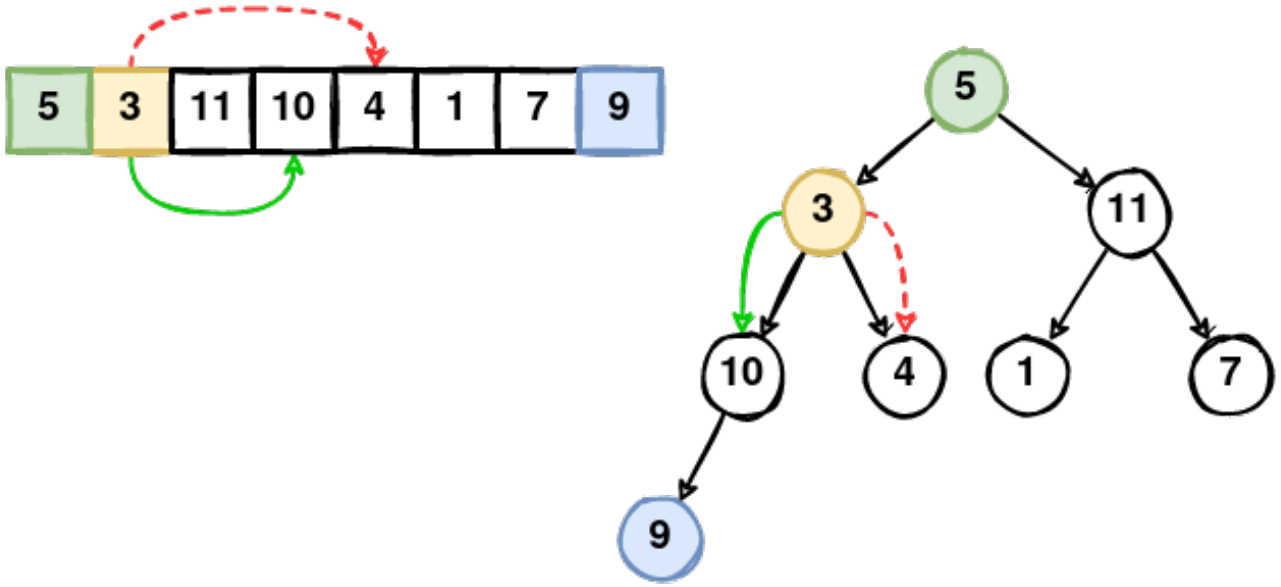
HeapSort: Build Heap



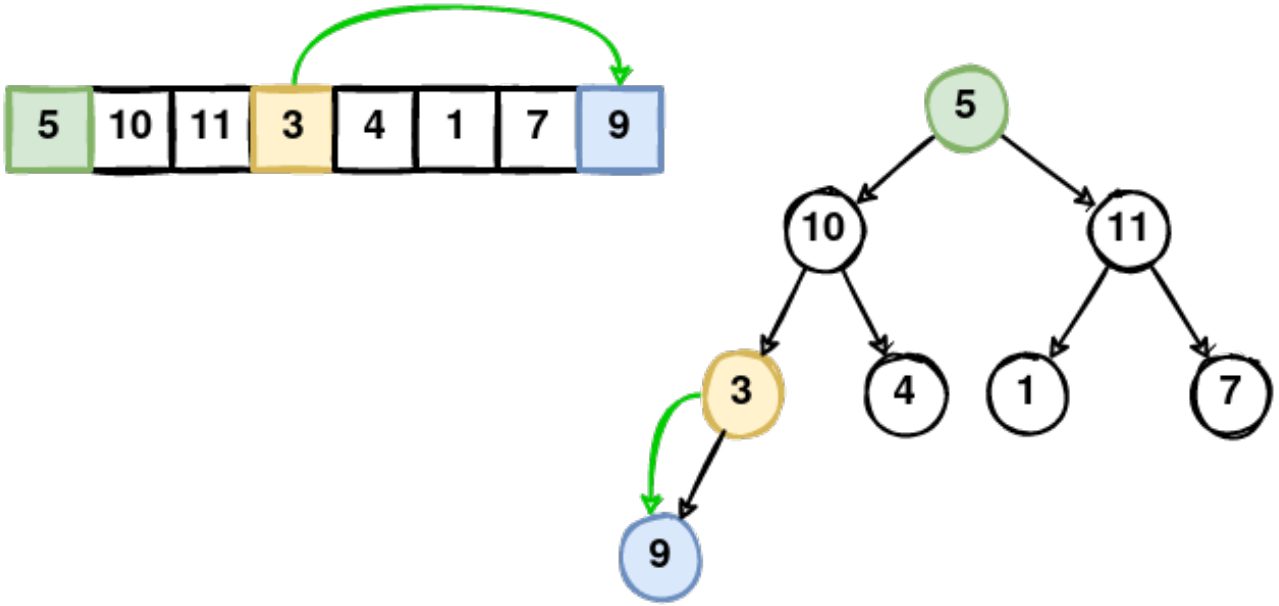
HeapSort: Build Heap



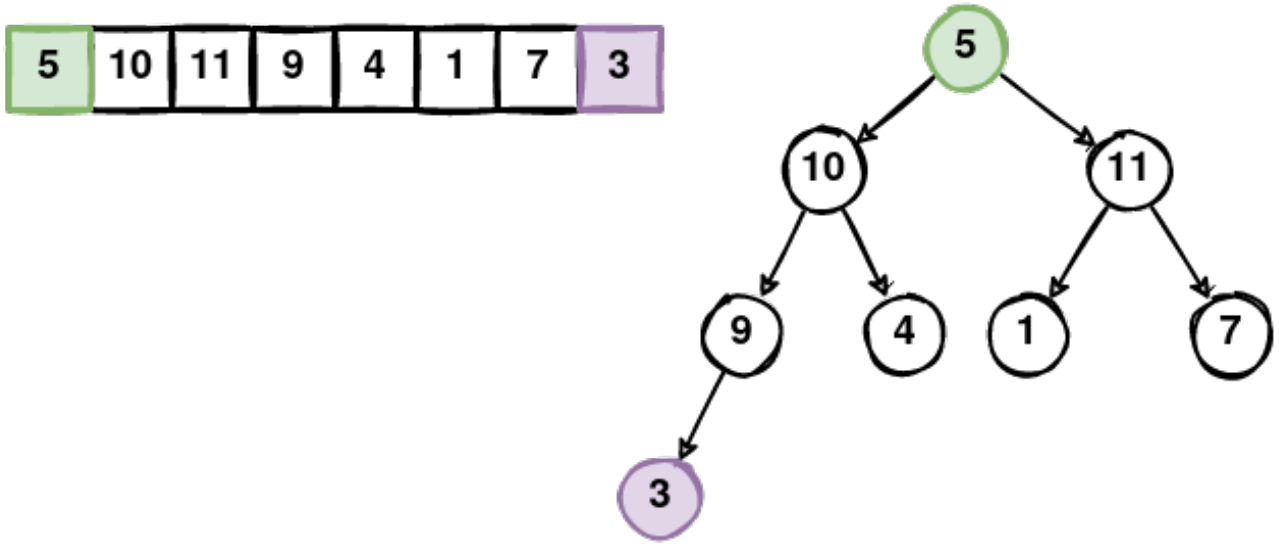
HeapSort: Build Heap



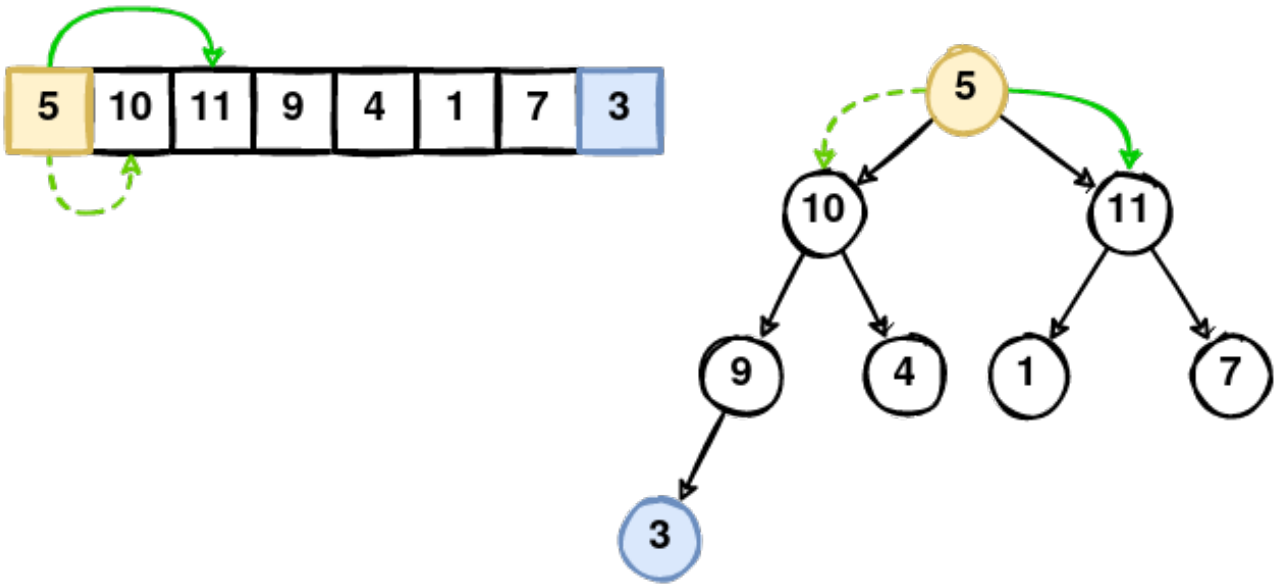
HeapSort: Build Heap



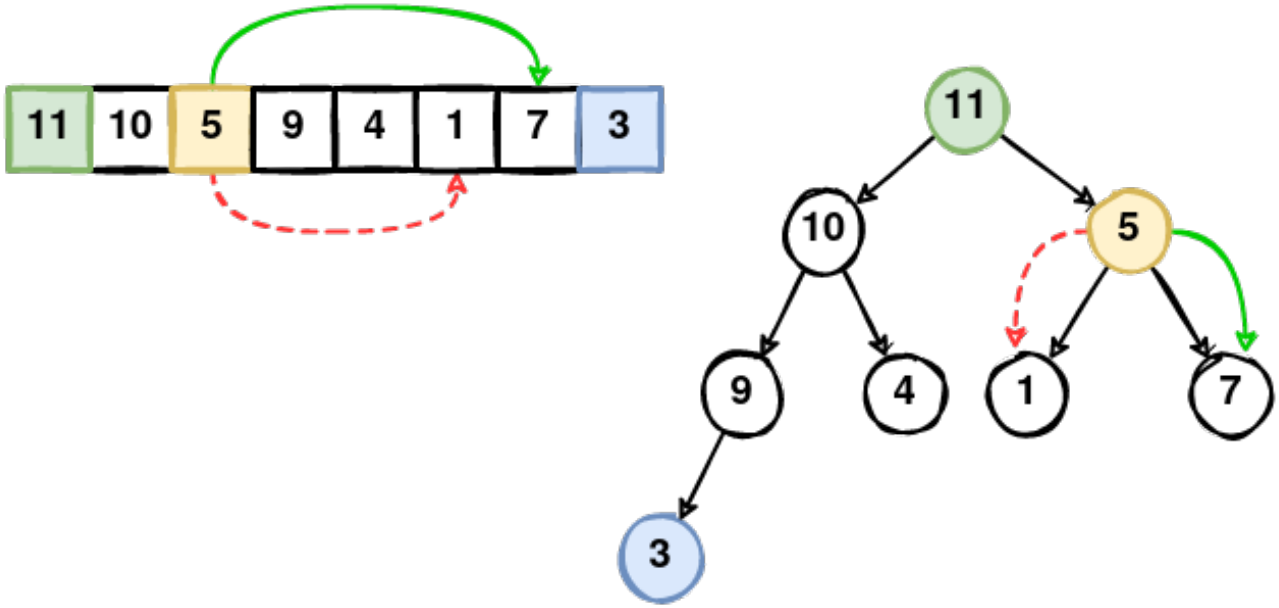
HeapSort: Build Heap



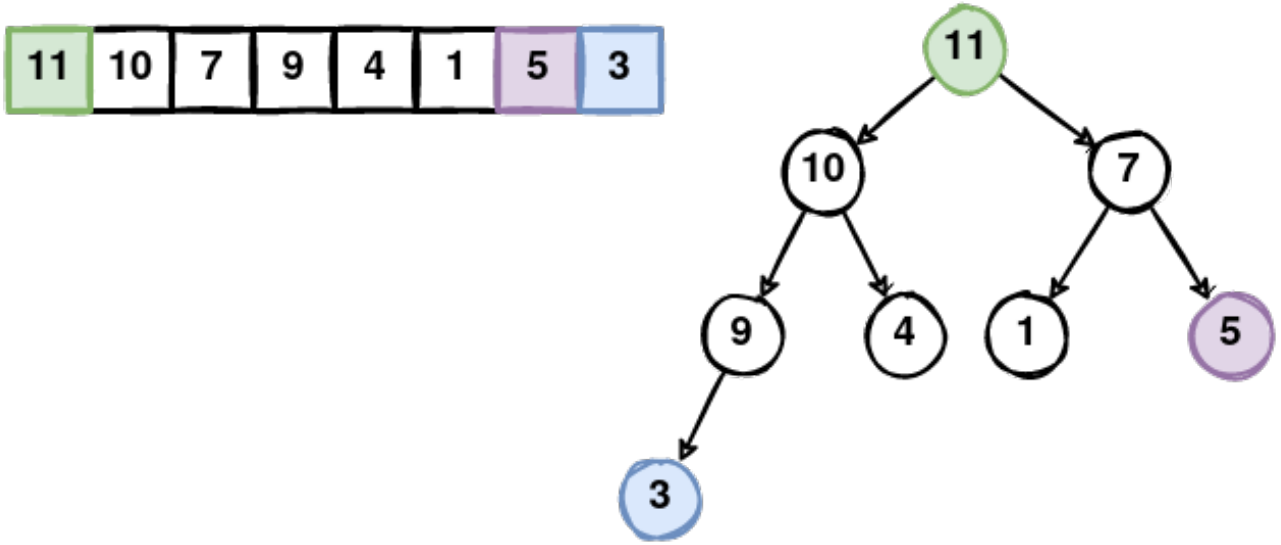
HeapSort: Build Heap



HeapSort: Build Heap



HeapSort: Build Heap

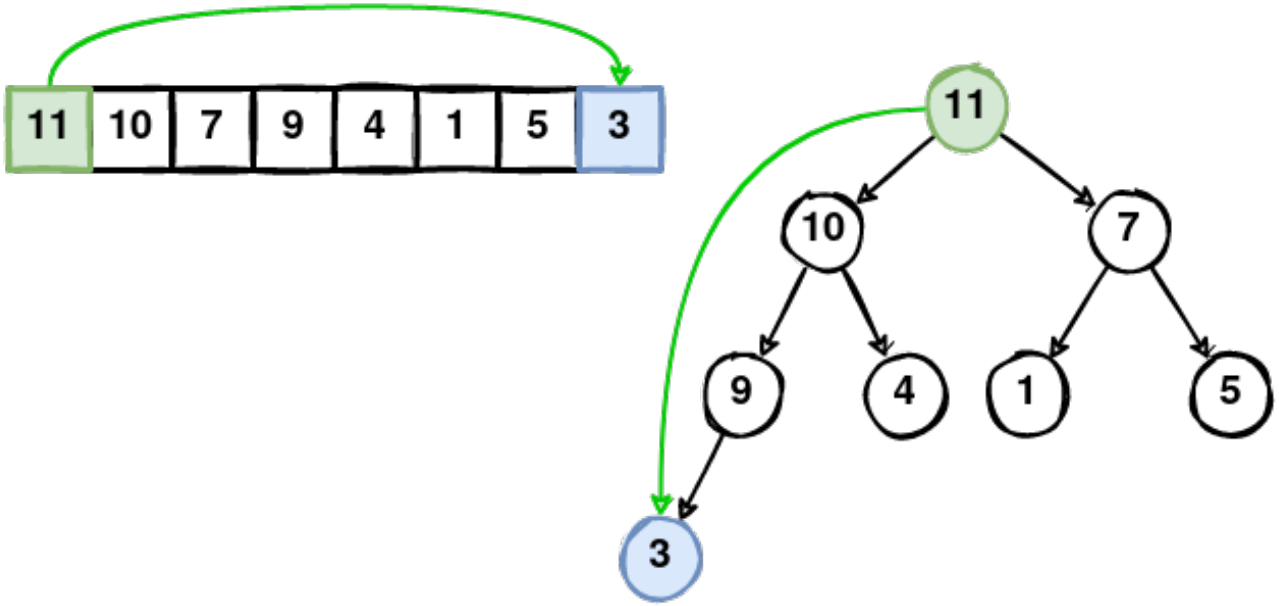


HeapSort: Build Heap

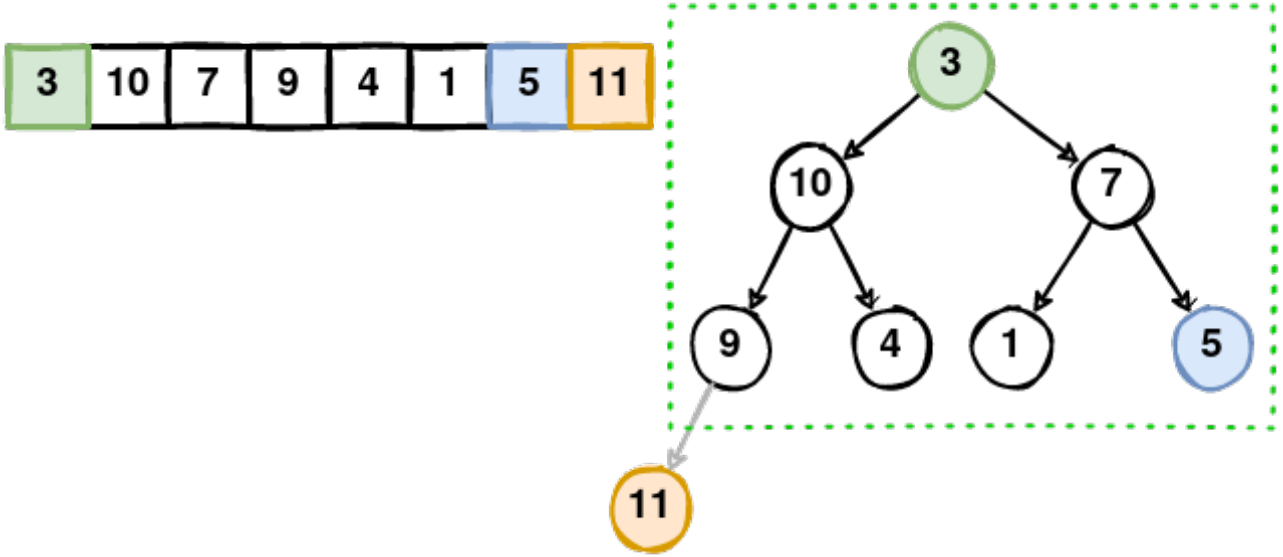
Build Heap

```
void build_heap(int arr[], int tam){ //arreglo y tamaño

        int i;
        for (i=tam/2; i>0; i--){
                heapify(arr, i, tam);
        }
}
```

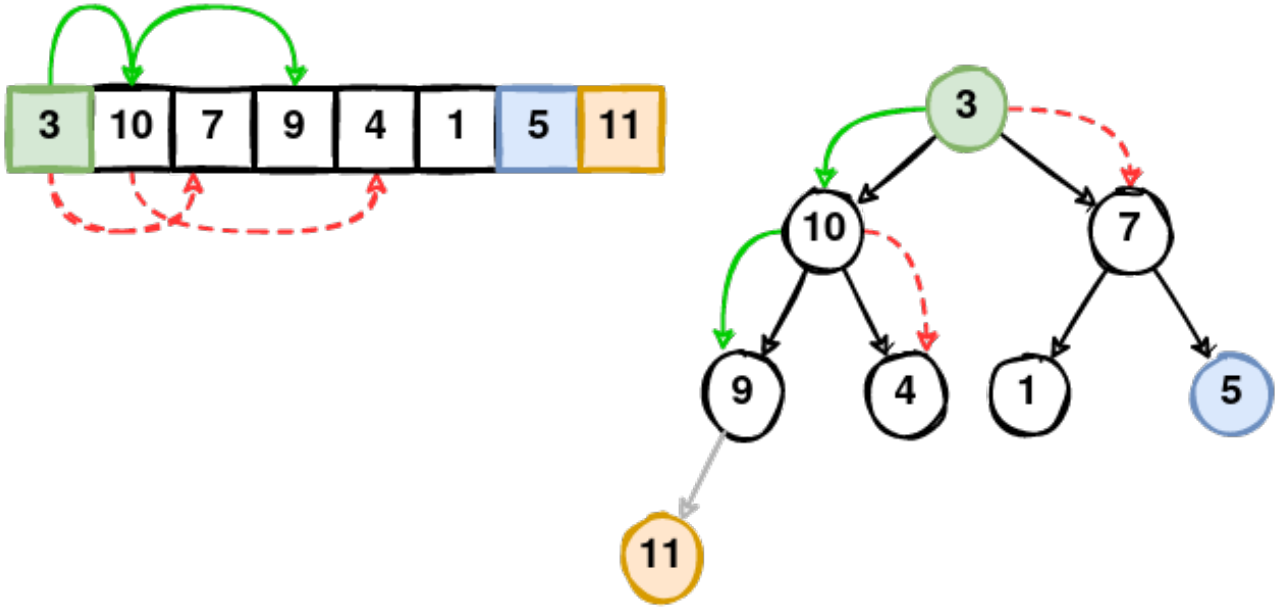
HeapSort



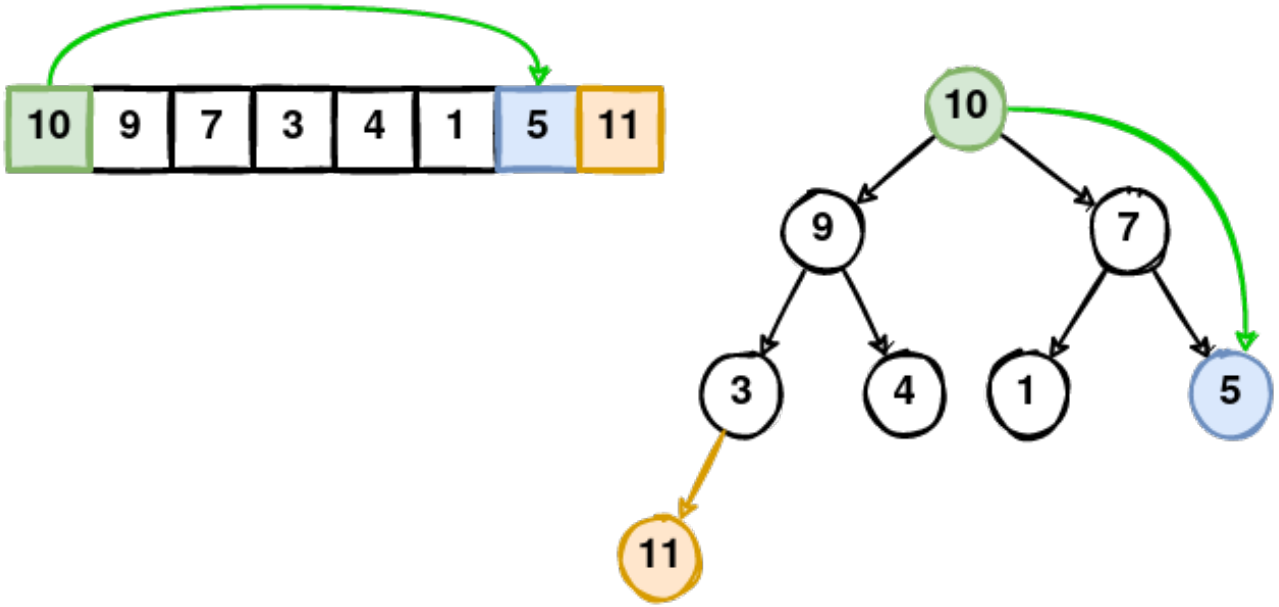
HeapSort



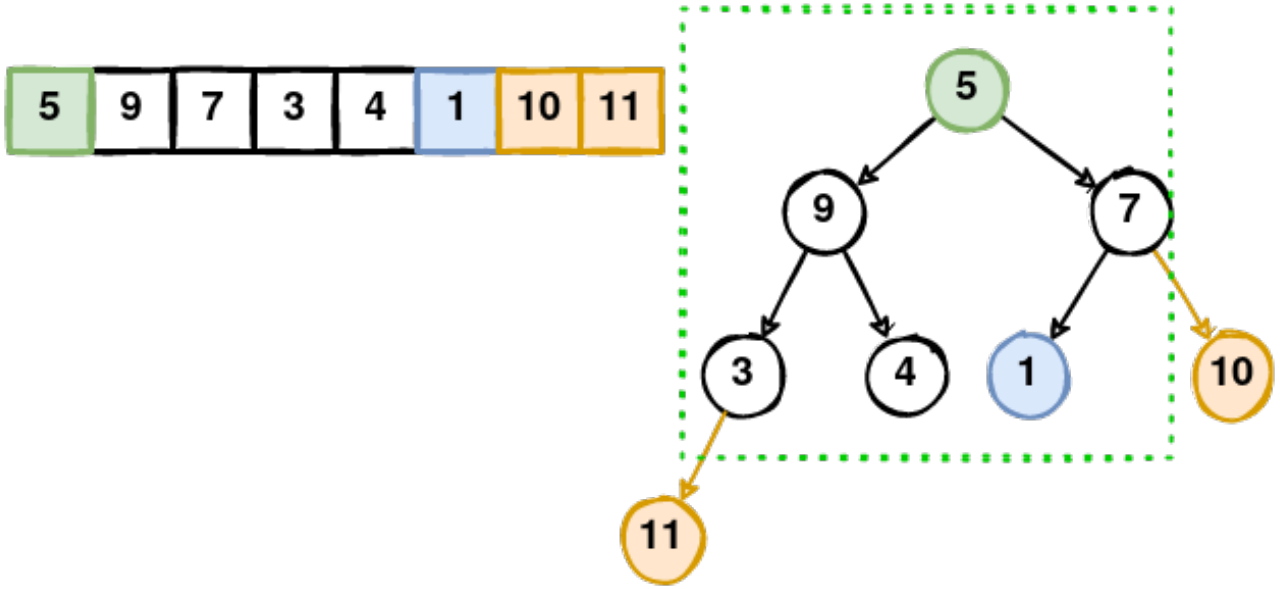
HeapSort



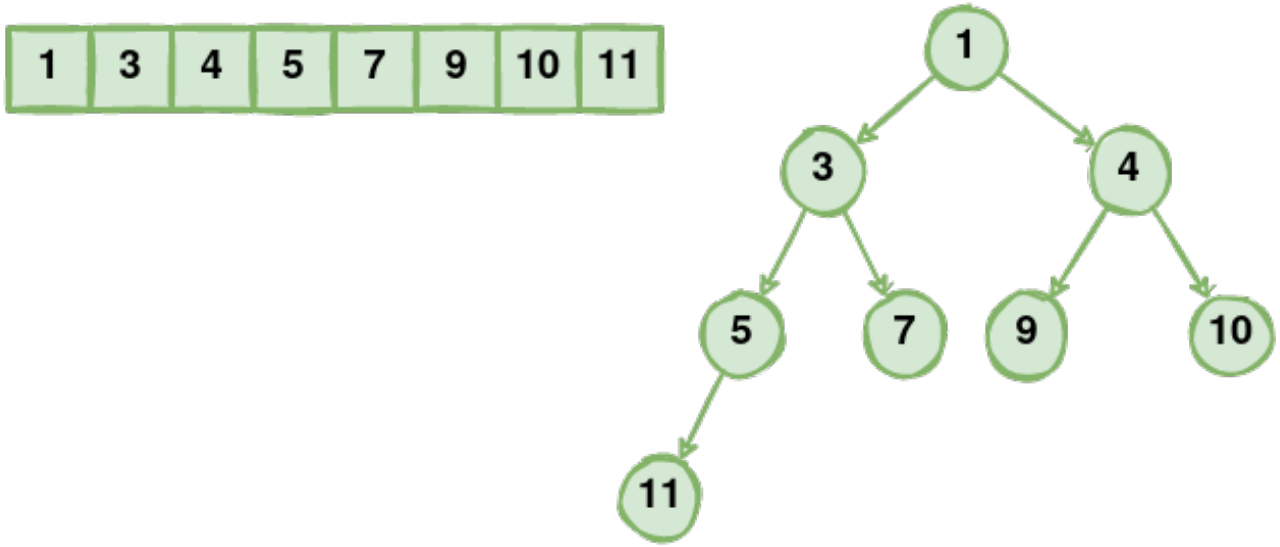
HeapSort



HeapSort



HeapSort



HeapSort

```
void HeapSort(int arr[], int tam){ //arreglo y tamaño

        int temp;

        build_heap(arr, tam);

        for (int i = tam; i > 1; i--){//cada vuelta el tamaño (i) baja en 1.
                temp=arr[1]; //dar vuelta primero con último
                arr[i] = arr[1];
                arr[1] = temp;

                heapify(arr, 1, i-1); //arreglar el heap post-cambio
        }

}
```

Colas de Prioridad

La estructura del heap (binary heap), es bastante utilizada, siendo su mayor aplicación las **Colas de Prioridad**.
En una cola de prioridad, datos son asociados con un valor (entero) representando su prioridad, y son ordenados según ésta.

- Schedulers de clúster (qsub, sbatch, etc).
- Prioridad de procesos (Task Manager, PID priority).

Se utiliza una cola con MAX-HEAP, la cual es fácilmente implementada en un arreglo.

Colas de Prioridad: Operaciones básicas

- `get_heap`: Retorna el valor del máximo valor en la cola.
- `pop_heap`: Retorna y remueve el valor máximo en la cola.
- `increase_key`: Actualiza el valor de prioridad de un elemento, y reordena el heap de tal forma de mantener sus reglas.
- `insert_key`: Añade un nuevo valor a la cola, y la ordena de tal forma de mantener las reglas del heap.

- `get_heap`: $\mathcal{O}(1)$.
- `pop_heap`: $\mathcal{O}(\log n)$.
- `increase_key`: $\mathcal{O}(\log n)$.
- `insert_key`: $\mathcal{O}(\log n)$.

HeapSort

- Dibuje e implemente el proceso de cada uno de los métodos de cola de prioridad.
- En un MAX-HEAP, asumiendo que todos los elementos son distintos, ¿dónde es posible encontrar el elemento con el menor valor y por qué?
- Un arreglo con todos sus elementos en orden, ¿puede ser un MIN-HEAP? ¿Por qué?
- Un arreglo con los siguientes valores $\{23; 17; 14; 6; 13; 10; 1; 5; 7; 12\}$ ¿puede ser un MAX-HEAP?
- Dibuje la operación de `heapify` en la posición 3 en el siguiente arreglo $A = \{27; 17; 3; 16; 13; 10; 1; 5; 7; 12; 4; 8; 9; 0\}$.
- Transcriba `heapify` para un heap que utiliza MIN-HEAP.
- ¿Cuál es el resultado de invocar a `heapify` cuando el elemento A[i] es mayor que sus hijos?
- Transcriba `heapify` usando ciclos sin recursividad.